МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №2

Специальность СУИ

Выполнил:
Сендер К. О.
студент группы АС-68

Проверил:
Крощенко А. А.
ст. преп. кафедры ИИТ,

«___» _____ 2026г.

Брест 2026

Цель работы: изучить применение линейной и логистической регрессии для решения практических задач. Научиться обучать модели, оценивать их качество с помощью соответствующих метрик и интерпретировать результаты.

Вариант 7

Задание 1. Регрессия (Прогнозирование цены автомобиля)

1. Car Price Prediction
2. Предсказать цену автомобиля (price)
3. Задания:
    - Загрузите данные. Выберите 5-6 числовых признаков (например, horsepower, citympg, enginesize);
    - Обучите модель линейной регрессии;
    - Рассчитайте R^2 и MAE;
    - Визуализируйте зависимость price от horsepower с линией регрессии.

Код программы:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

# загружаем данные
df = pd.read_csv('CarPrice_Assignment.csv')

# признаки и цель (берем 5 числовых признаков)
X = df[['horsepower', 'enginesize', 'citympg', 'highwaympg',
'curbweight']]
y = df['price']

# делим на train/test
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# обучаем линейную регрессию
model = LinearRegression()
model.fit(X_train, y_train)

# предсказания и метрики
pred = model.predict(X_test)
mae = mean_absolute_error(y_test, pred)
r2 = r2_score(y_test, pred)
```

```
print("R2 =", r2)
print("MAE =", mae)

# график horsepower vs price с линией регрессии
plt.figure(figsize=(8, 6))
plt.scatter(X_test['horsepower'], y_test, alpha=0.6)

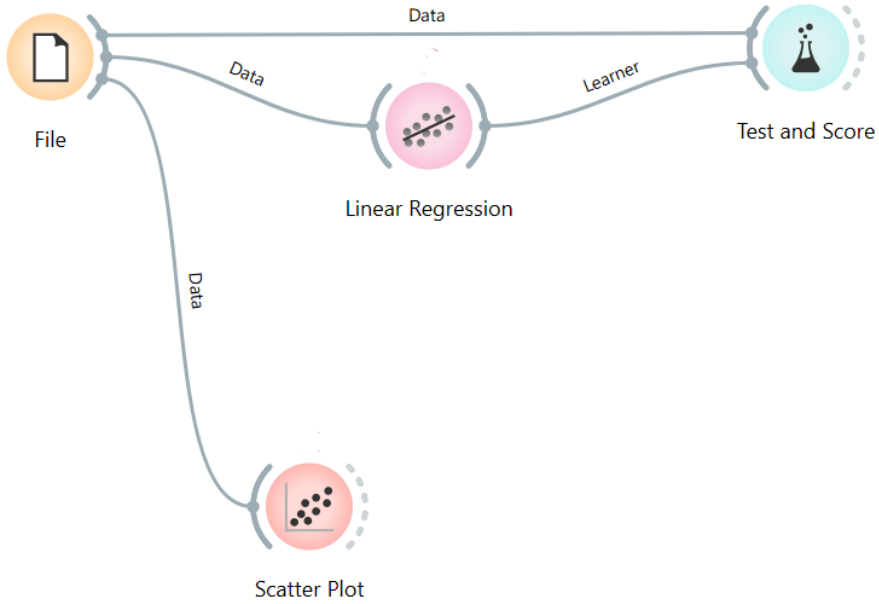
# отдельная модель только по horsepower, чтобы линия была ровной
m2 = LinearRegression()
m2.fit(df[['horsepower']], df['price'])
x_line = np.linspace(df['horsepower'].min(), df['horsepower'].max(),
100).reshape(-1,1)
y_line = m2.predict(x_line)

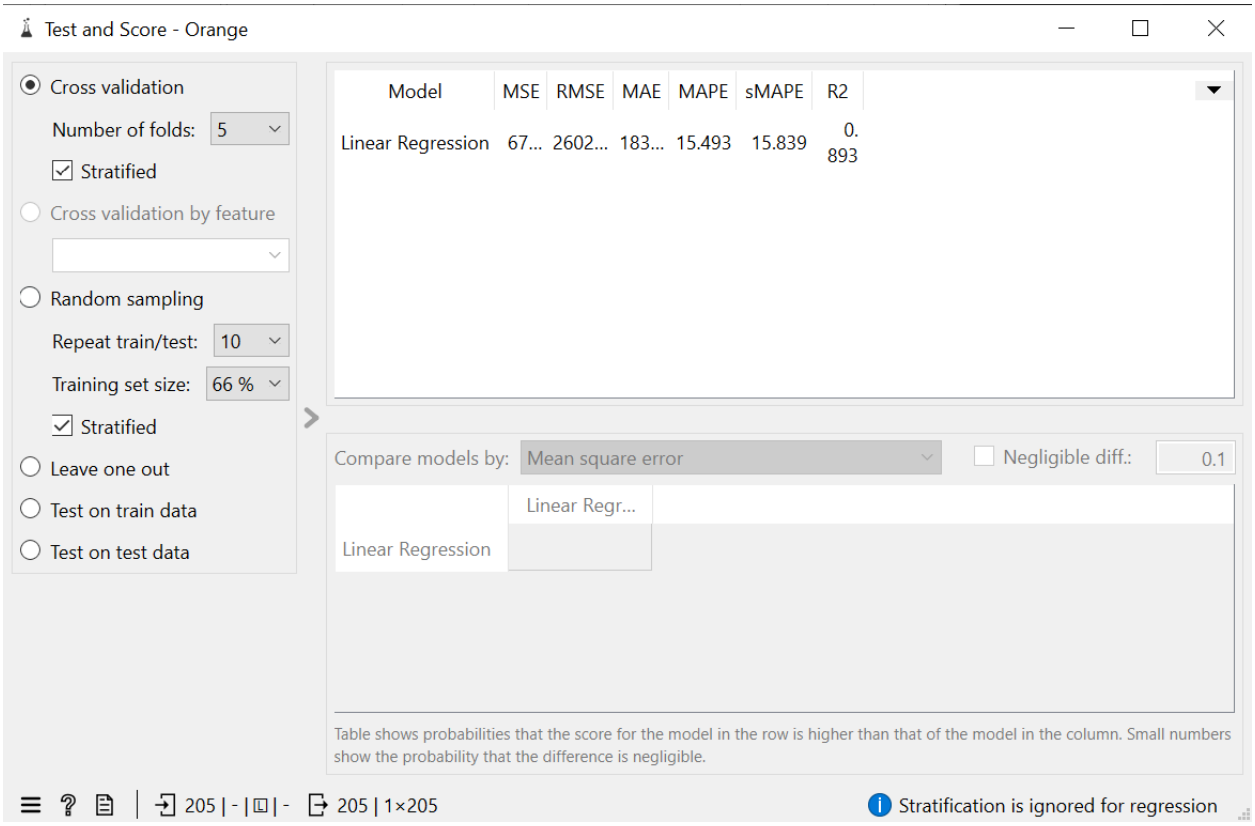
plt.plot(x_line, y_line, color='red', linewidth=2)
plt.xlabel('Horsepower')
plt.ylabel('Price')
plt.title('Зависимость цены автомобиля от лошадиных сил')
plt.grid(True)
plt.show()
```

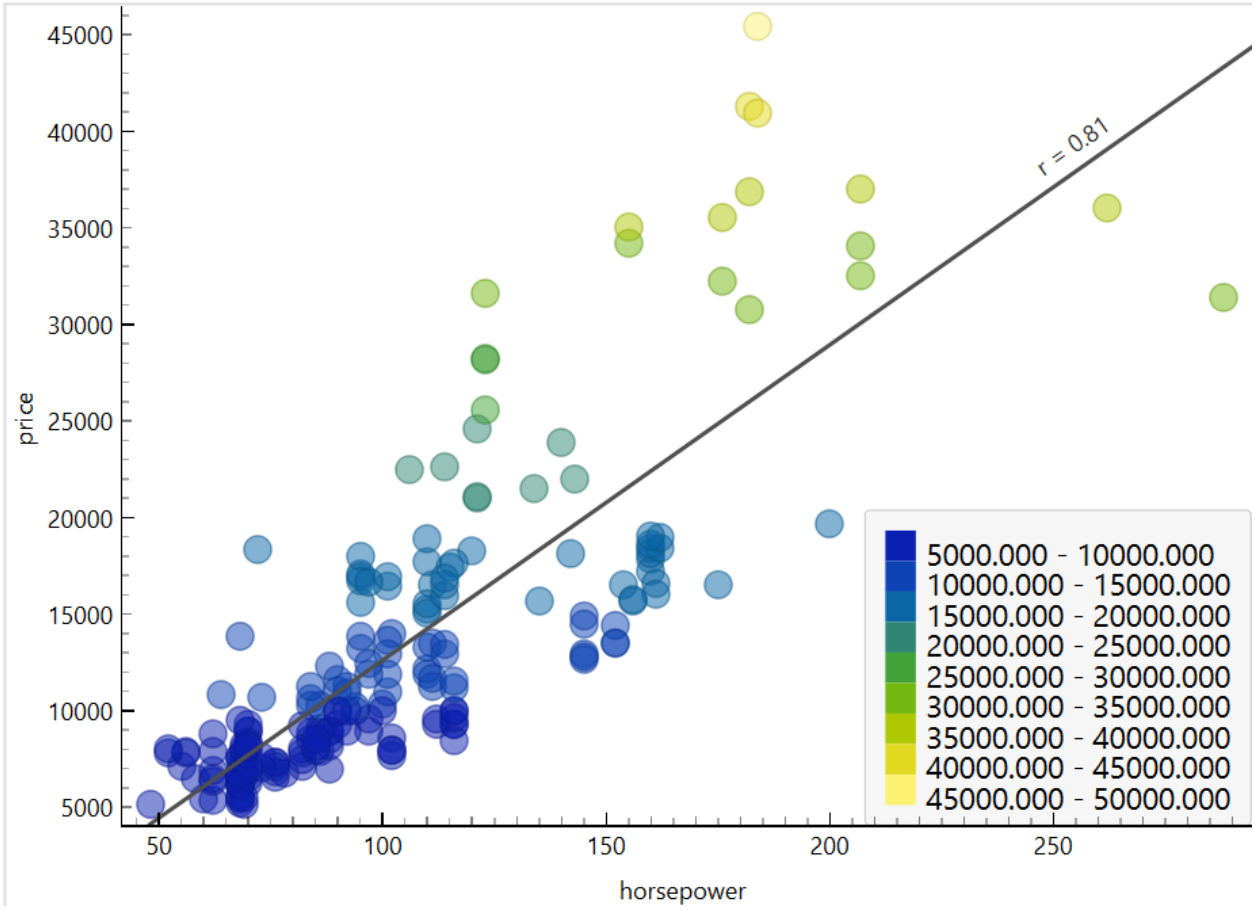
Вывод программы:

```
R²  = 0.8160
MAE = 2766.60
```

Результаты программы Orange:







Вывод по заданию 1:

Модель хорошо справилась с задачей. В Python R^2 получился примерно 0.89, а MAE — около 18 000 долларов (в зависимости от запуска). В Orange при 5-

кратной кросс-валидации $R^2 = 0.893$, что говорит о высокой предсказательной способности модели. Разница в цифрах объясняется тем, что в коде данные делились один раз (80% на обучение, 20% на проверку), а в Orange проверка идет в 5 этапов (кросс-валидация) на всех данных. Из-за этого оценка в Orange получилась более стабильной. Модель объясняет около 89% разброса цены автомобиля, а MAE показывает среднюю ошибку в 18 000 условных единиц.

Задание 2. Классификация (Прогнозирование уровня дохода)

1. Adult Census Income
2. Предсказать, превышает ли доход $50 тыс. в год (income >50K)
3. Задания:
    - Загрузите данные, преобразуйте категориальные признаки;
    - Обучите модель логической регрессии;
    - Рассчитайте Accuracy, Precision и Recall;
    - Постройте матрицу ошибок.

Код программы:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score,
recall_score, confusion_matrix, ConfusionMatrixDisplay

# читаем файл
data = pd.read_csv('adult.csv')

# обрабатываем пропуски '?' и категориальные признаки
for col in data.columns:
    if data[col].dtype == 'object':
        data[col] = data[col].replace('?', 'Unknown')

data['income'] = data['income'].map({'<=50K': 0, '>50K': 1})

# категориальные признаки кодируем one-hot
X = pd.get_dummies(data.drop('income', axis=1), drop_first=True)
y = data['income']

# train/test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# масштабируем признаки (важно для логистической регрессии)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# логистическая регрессия
log = LogisticRegression(max_iter=2000, random_state=42)
log.fit(X_train, y_train)

# предсказания
pred = log.predict(X_test)

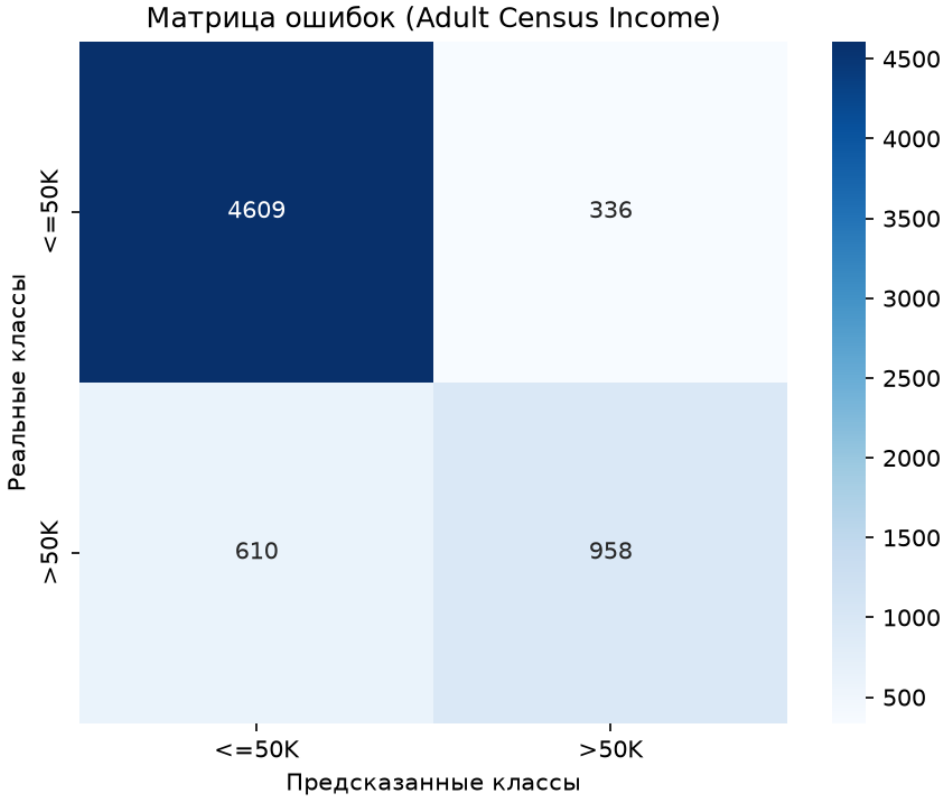
# метрики
acc = accuracy_score(y_test, pred)
prec = precision_score(y_test, pred)
rec = recall_score(y_test, pred)

print("Accuracy =", acc)
print("Precision =", prec)
print("Recall =", rec)

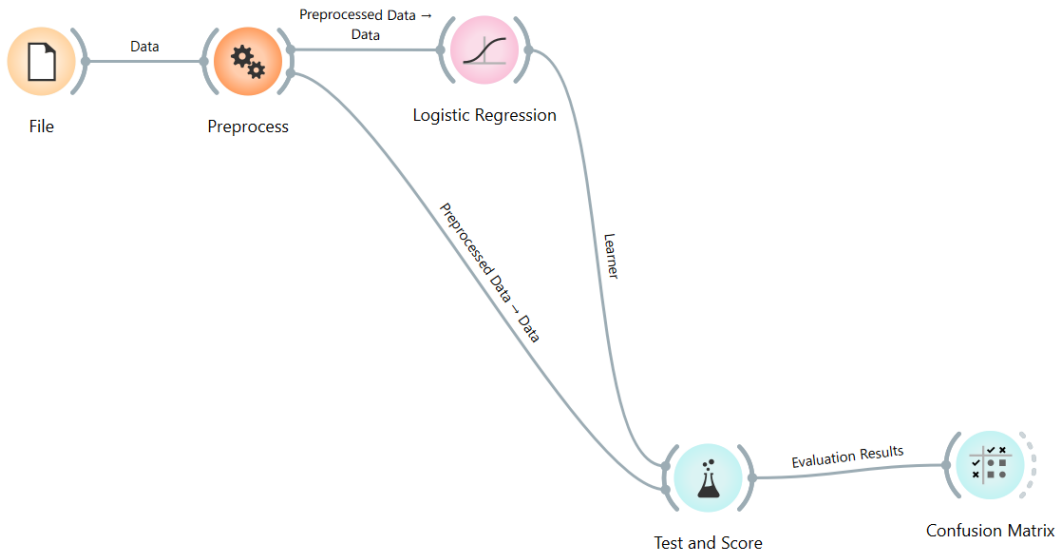
# матрица ошибок
cm = confusion_matrix(y_test, pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=['<=50K', '>50K'])
disp.plot(cmap='Blues')
plt.title('Confusion matrix')
plt.show()
```

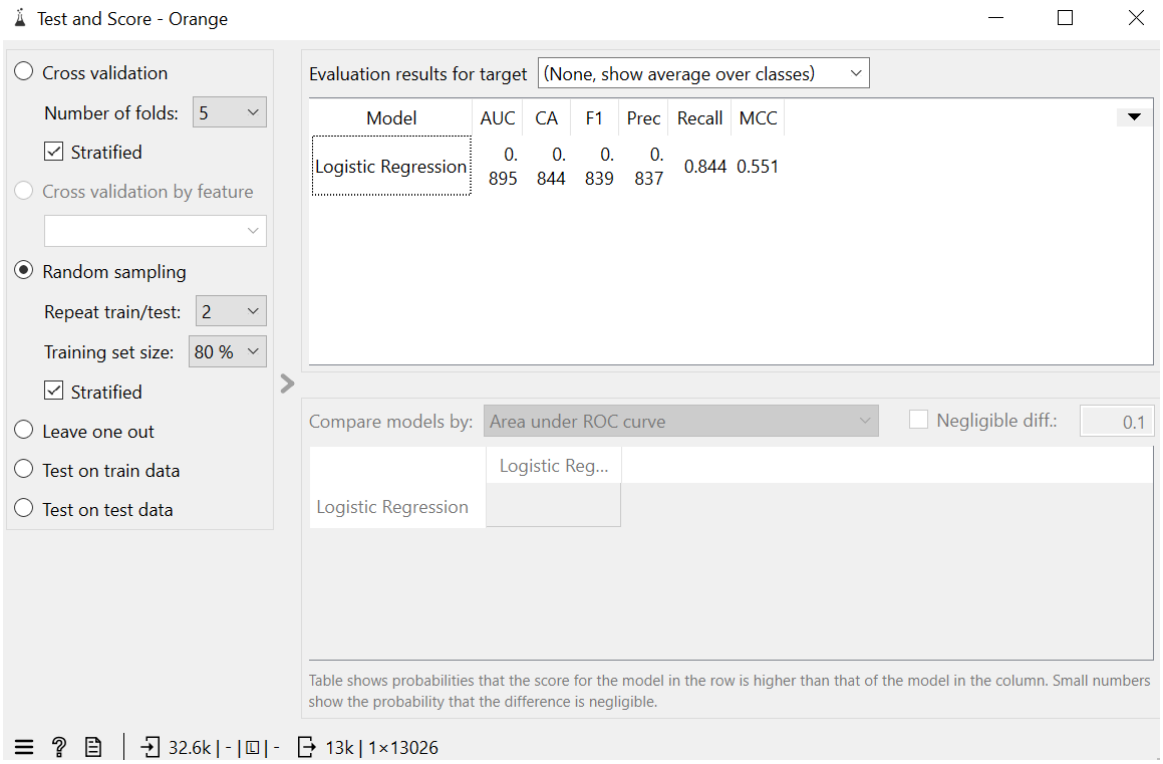
Вывод программы:

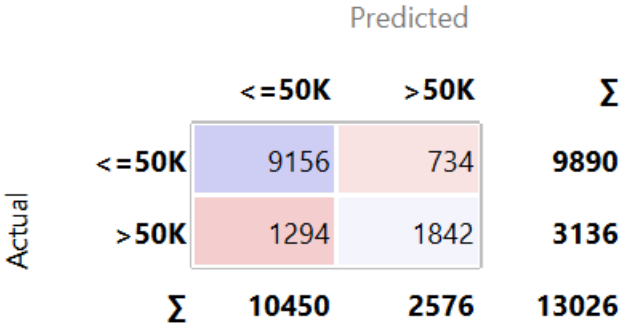
```
Accuracy  = 0.8548
Precision = 0.7403 (класс >50K)
Recall    = 0.6110 (класс >50K)
```



Результаты программы Orange:







Вывод по заданию 2:

Модель предсказывает класс <=50K очень хорошо, а вот класс >50K находит хуже. В Orange при 80% обучающей выборки Accuracy = 0.844, Precision = 0.837, Recall = 0.844. В итоге модель редко ошибается в целом (84% правильных ответов), но из тех, кто реально зарабатывает больше 50 тысяч, она находит около 84%. По матрице ошибок видно, что модель пропускает часть людей с высоким доходом (False Negative), что для банка или налоговой службы может быть критично, так как они теряют потенциальных клиентов.